

Real-time Image Analysis for Self Driving Capabilities

Muhammad Muhtashim, Muhammad Taha, Muhammad Sohaib Afzal, Muhammad Hashir Ashraf Awan

Department of Computer & Software Engineering, College of E&ME, NUST, Rawalpindi, Pakistan

Course: Digital Image Processing

Submitted to: Mam LE Sundas Ashraf, Dr Asad Khan

Abstract—This paper presents a real-time image analysis system for self-driving capabilities that integrates lane highlighting, red-object obstacle detection, and rule-based directional decision-making. The system processes a video stream frame-by-frame using OpenCV and NumPy, producing an annotated output with highlighted drivable lane regions and detected obstacles, while simulating navigation decisions in real time.

Index Terms—Lane detection, obstacle detection, OpenCV, HSV segmentation, connected components, autonomous navigation.

I. INTRODUCTION

Autonomous navigation requires reliable real-time perception to ensure safe motion. This work implements a real-time pipeline that detects the most prominent lane region, identifies red-colored obstacles, and determines feasible motion directions based on free space. The system targets driving assistance and robotics scenarios where fast decisions are needed from streaming visual input.

II. SYSTEM OVERVIEW

The system is implemented in Python using OpenCV and NumPy and performs:

- **Lane Detection:** highlighting the most prominent lane region using edge processing and connected component analysis.
- **Obstacle Detection:** detecting red-colored obstacles via HSV thresholding and contour filtering.
- **Directional Decision Making:** choosing among forward, right, left, or backward based on lane space availability.

III. METHODOLOGY

A. Lane Highlighting

Lane highlighting is achieved by applying Canny edge detection followed by morphological closing to connect discontinuities. The lower half of the frame is then thresholded and analyzed using connected components. A scoring function selects the best lane candidate using:

- minimum area threshold,
- edge pixel density within the component,
- proximity to the horizontal image center,
- preference for vertically extended components.

The selected lane is overlaid on the frame (yellow fill).

B. Obstacle Detection

Obstacle detection uses HSV color segmentation to isolate red-colored regions:

- Convert the frame from BGR to HSV.
- Threshold using a fixed HSV range for red.
- Extract contours and ignore small regions using an area threshold.
- Draw bounding boxes around valid detections.

C. Directional Decision Making

Directional decision-making evaluates whether the simulated agent can move within the highlighted lane region. The algorithm checks pixels in the intended direction for the lane color and selects the first valid movement among:

- 1) forward,
- 2) right,
- 3) left,
- 4) backward.

If movement is possible, the agent coordinates are updated and the agent is rendered on the processed frame.

D. Real-Time Video Processing

For each video frame:

- 1) Read and resize frame.
- 2) Detect obstacles and draw bounding boxes.
- 3) Highlight lane region and produce edge map.
- 4) Compute movement decision and update simulated agent.
- 5) Display the edge map and final annotated frame.

IV. IMPLEMENTATION

Key functions include obstacle detection, lane highlighting, decision making, and the real-time processing loop.

```
import cv2
import numpy as np

def detect_obstacles(frame):
    hsv_frame = cv2.cvtColor(frame, cv2.
        COLOR_BGR2HSV)

    color_range_min = np.array([0, 45, 50])
    color_range_max = np.array([15, 255, 255])

    obstacle_mask = cv2.inRange(hsv_frame,
        color_range_min, color_range_max)
```

```

contours, _ = cv2.findContours(obstacle_mask,
                               cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

detected_boxes = []
for cnt in contours:
    if cv2.contourArea(cnt) > 150:
        x, y, w, h = cv2.boundingRect(cnt)
        detected_boxes.append((x, y, w, h))
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(frame, "Object", (x, y - 8),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.5, (0, 255, 0), 2)

return frame, detected_boxes

def highlight_largest_lane_region(frame,
    edge_pixel_thresh=100):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 50, 150)

    kernel = np.ones((5, 5), np.uint8)
    closed_edges = cv2.morphologyEx(edges, cv2.
        MORPH_CLOSE, kernel)
    inv = cv2.bitwise_not(closed_edges)

    h, w = inv.shape
    bottom = inv[h // 2:, :]
    _, binary = cv2.threshold(bottom, 127, 255, cv2.
        THRESH_BINARY)

    num_labels, labels, stats, centroids = cv2.
        connectedComponentsWithStats(binary,
        connectivity=8)

    min_area = 4000
    best_label = -1
    best_score = 0

    for i in range(1, num_labels):
        x, y, cw, ch, area = stats[i]
        if area < min_area:
            continue

        component_mask = (labels == i).astype(np.
            uint8) * 255
        edge_pixels_in_component = cv2.bitwise_and(
            bottom, bottom, mask=component_mask)
        nonzero_edge_count = cv2.countNonZero(
            edge_pixels_in_component)

        if nonzero_edge_count < edge_pixel_thresh:
            continue

        cx, cy = centroids[i]
        dist_center = abs((w / 2) - cx)
        score = area - 5 * dist_center + ch

        if score > best_score:
            best_score = score
            best_label = i

    if best_label == -1:
        return frame, edges, frame

    mask = np.zeros_like(binary)
    mask[labels == best_label] = 255

    full_mask = np.zeros_like(gray)
    full_mask[h // 2:, :] = mask

    contours, _ = cv2.findContours(full_mask, cv2.
        RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    overlay = frame.copy()
    for cnt in contours:

```

```

        cv2.drawContours(overlay, [cnt], -1, (0,
            255, 255), -1)

    blended = cv2.addWeighted(overlay, 0.4, frame,
        0.6, 0)
    return overlay, edges, blended

def directional_decision_making(road,
    highlighted_frame, obj, x1, x2, y1, y2):
    highlighted_frame[x1:x2, y1:y2] = obj

    check_upward, check_right, check_left,
        check_backward = True, True, True, True

    for i in range(y1, y2):
        if list(road[x1 - 1, i]) != [0, 255, 255]:
            check_upward = False

    if check_upward:
        x1 -= 1; x2 -= 1
    else:
        for i in range(x1, x2):
            if list(road[i, y2 + 1]) != [0, 255,
                255]:
                check_right = False

    if check_right or list(road[i, y1 - 20]) !=
        [0, 255, 255]:
        y1 += 1; y2 += 1
    else:
        for i in range(x1, x2):
            if list(road[i, y1 - 1]) != [0, 255,
                255]:
                check_left = False

    if check_left or list(road[i, y2 + 20])
        != [0, 255, 255]:
        y1 -= 1; y2 -= 1
    else:
        for i in range(y1, y2):
            if list(road[x1 + 1, i]) != [0,
                255, 255]:
                check_backward = False

    if check_backward or list(road[x1 -
        20, i]) != [0, 255, 255]:
        x1 += 1; x2 += 1

    return highlighted_frame, x1, x2, y1, y2

def process_video_realtime(video_path):
    cap = cv2.VideoCapture(video_path)
    obj = np.ones((20, 20, 3), dtype=np.uint8) * np.
        array([0, 0, 255], dtype=np.uint8)
    x1, x2 = 400, 420
    y1, y2 = 240, 260

    if not cap.isOpened():
        print("Failed to open video.")
        return

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        frame = cv2.resize(frame, (640, 480))

        frame, _ = detect_obstacles(frame)
        overlay, edge_map, highlighted_frame =
            highlight_largest_lane_region(frame)
        frame, x1, x2, y1, y2 =
            directional_decision_making(
                overlay, highlighted_frame, obj, x1, x2,

```



Fig. 1. Sample output showing obstacle highlighting and road/lane detection.

```

        y1, y2
    )

    cv2.imshow("Edge Map", edge_map)
    cv2.imshow("Lane and Obstacle Detection",
               frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    cap.release()
    cv2.destroyAllWindows()

```

V. RESULTS AND EVALUATION

The proposed system successfully highlights the dominant lane region and detects red-colored obstacles in real-time video frames. The decision module updates the agent position based on the availability of lane-colored free space. Observed outcomes include:

- **Lane Highlighting:** robust under partial occlusions due to connected component scoring.
- **Obstacle Detection:** effective for red objects with area-based filtering.
- **Decision Making:** provides a simple but functional rule-based navigation simulation.

A. Output Link

Pre-recorded video check: https://drive.google.com/drive/folders/1utqxc7IJvs9J0k3RKJoTLrGlA9ayWy7?usp=drive_link

VI. SAMPLE OUTPUT

Fig. 1 shows a representative output frame where the system highlights the road/lane region and detects obstacles.

VII. CONCLUSION

A real-time pipeline for lane highlighting, red-object obstacle detection, and rule-based movement decisions was implemented using OpenCV and NumPy. The approach is suitable for real-time demonstrations and can be extended by incorporating multi-color/multi-class detection, robust lane modeling, and learning-based decision modules.